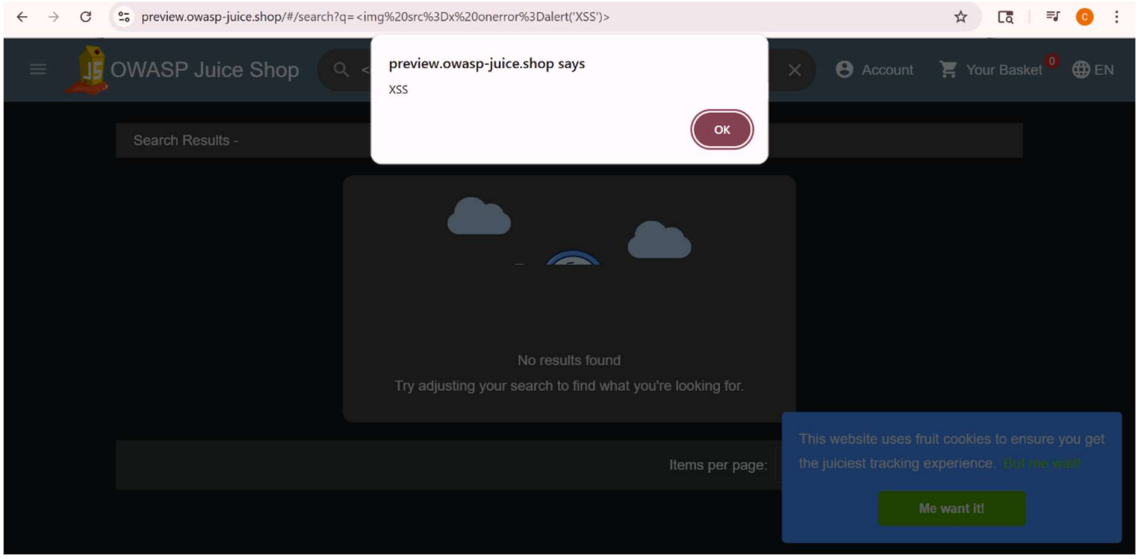
SECTION 6: JavaScript & XSS

Task 6.1: XSS on Juice Shop

1. **Basic alert**
   Payload: *<script>alert('XSS')</script>*
   Explanation:
   This payload attempts to inject JavaScript using a <script> tag to display an alert box. In this test, the payload did not execute. This is likely because the application sanitizes or blocks direct script tags as a security measure. This shows that basic XSS filtering is present, but other payloads using event handlers may still bypass the protection.

2. **Image tag**



Payload: *<img src=x onerror=alert('XSS')>*

Explanation:

This payload uses an invalid image source (src=x). Since the image cannot load, the browser triggers the onerror event. The JavaScript inside the event runs and displays an alert box saying "XSS".

This proves that the search feature in OWASP Juice Shop is vulnerable to Cross-Site Scripting (XSS), because user input is executed without proper validation.

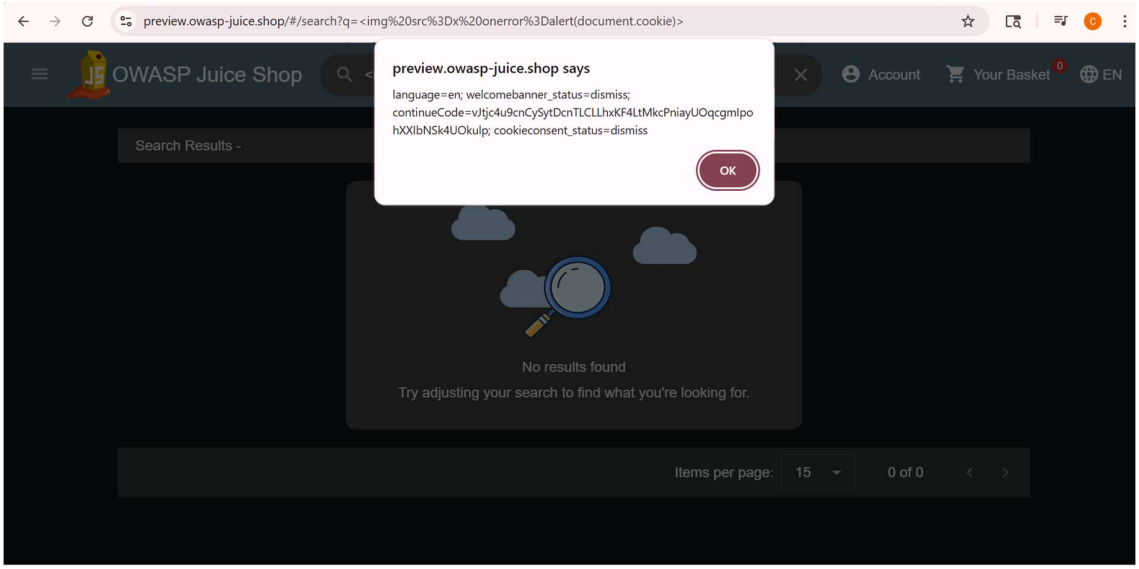
3. **SVG**

   Payload: *<svg/onload=alert('XSS')>*

   Explanation:

   This payload uses the SVG tag with the onload event to execute JavaScript when the element loads. In this test, the payload did not execute. This is likely because the application filters or sanitizes SVG-based input to prevent script execution. This shows that some XSS protections are in place, although other payloads may still bypass them.

4. **Cookie display**



   Payload: *<img src=x onerror=alert(document.cookie)>*

   Explanation:

   This payload attempts to display the browser's cookies using JavaScript when the image fails to load. It demonstrates how attackers can use XSS vulnerabilities to access session cookies and potentially steal user session information. This type of attack can lead to session hijacking or unauthorized access if sensitive cookies are exposed.

5. **DOM manipulation**

Payload: *<img src=x onerror="document.body.innerHTML='HACKED'">*

Explanation:

This payload uses the onerror event of an invalid image to execute JavaScript when the image fails to load. The script replaces the entire webpage content with the word “HACKED”.

This demonstrates how XSS can be used to modify the Document Object Model (DOM), change webpage content, and affect the user experience. Such attacks can be used to deface websites, mislead users, or perform malicious actions.